



Universidad Latina de Costa Rica

Curso: Sistemas Operativos

Tema: Análisis Comparativo del Rendimiento, Seguridad y Gestión de Recursos
en los Sistemas Operativos Modernos

Integrantes del Grupo:

Byron Jimenez Isaacs

Ronald Vargas González

Profesor: Carlos González Jara

Fecha de Entrega: 14 de agosto 2026

II Cuatrimestre del 2026

Índice

Índice	1
Resumen ejecutivo	2
1. Planteamiento del problema	3
2. Justificación	4
3. Objetivo general	5
4. Objetivos específicos	6
5. Preguntas de investigación	6
6. Hipótesis	7
7. Marco teórico	8
7.1 Evolución de los sistemas operativos	9
7.2 Arquitectura de un sistema operativo	9
7.3 Administración de procesos y planificación del CPU	10
7.4 Memoria principal y memoria virtual	11
7.5 Sistemas de archivos	12
7.6 Seguridad informática	13
7.7 Virtualización	13
7.8 Evaluación del rendimiento	14
7.9 Cómputo distribuido y balanceo de carga	15
7.10 Concurrencia vs. paralelismo y el Global Interpreter Lock (GIL)	16
8. Metodología	17
8.1 Tipo de investigación y enfoque	18
8.2 Técnicas de recolección de datos	18
8.3 Instrumentos	18
8.4 Variables	19

8.5 Recursos	20
Humanos	21
Materiales	21
9. Sección práctica: diseño e implementación de OS Grid	21
9.1 Qué es OS Grid	22
9.2 Arquitectura del sistema	22
9.3 Stack tecnológico	23
9.4 Estructura del repositorio	24
9.5 Protocolo de comunicación.....	24
9.6 Máquina de estados del coordinador	25
9.7 Reparto de carga: work-stealing dinámico	26
9.8 Diseño experimental: las tres pruebas de cómputo	27
9.8.1 hash — fuerza bruta de SHA-256	27
9.8.2 sort — ordenamiento en memoria	27
9.8.3 primos — criba de Eratóstenes segmentada	28
9.9 Las tres intensidades	28
9.10 Medición de métricas.....	29
9.11 Persistencia de datos	30
9.12 Dashboard	31
9.12.1 Pestaña «Panel principal»	32
9.12.2 Pestaña « ⚡ Interacción SO» (vista Battle Royale).....	34
10. Tolerancia a fallos.....	37
11. Informe de pruebas y depuración.....	38
11.1 Casos de prueba ejecutados y su resultado	42
12. Resultados	44
12.1 Gráficas sugeridas a partir del CSV exportado	45

13. Análisis y discusión	46
13.1 macOS domina el rendimiento en bruto	47
13.2 Windows superó a Linux en esta prueba	47
13.3 Varianza	47
13.4 Uso de memoria (RAM)	48
13.5 CPU del proceso relativamente baja (6 % a 10 %)	48
13.6 Limitaciones metodológicas	49
14. Conclusiones.....	49
Anexos	51
Anexo A. Comandos de despliegue.....	52
Anexo B. Esquema SQL.....	52
Anexo C. Guía de reproducción.....	53

Resumen ejecutivo

Este informe presenta un análisis comparativo del rendimiento, la administración de recursos y la seguridad de los sistemas operativos Linux, macOS y Windows, combinando una revisión teórica de sus arquitecturas con un experimento práctico propio: OS Grid, un clúster de cómputo distribuido heterogéneo en el que un coordinador reparte trabajo entre tres equipos, cada uno corriendo un sistema operativo distinto, ejecutando exactamente el mismo código Python. El sistema implementa un mecanismo de reparto dinámico de tipo work-stealing, tolerancia a fallos mediante reencolado y vigilancia por timeout, persistencia de resultados en SQLite y un panel de control web en Angular con telemetría en vivo. Sobre catorce misiones ejecutadas con los tres sistemas operativos compitiendo simultáneamente, macOS obtuvo el mejor tiempo promedio por chunk (0.504 s), seguido de Windows (1.386 s) y Linux (1.680 s), con macOS ganador en las catorce corridas. El informe documenta también el proceso de depuración del sistema, en el que se identificaron y corrigieron ocho defectos que en su forma original podían trabar el coordinador de manera indefinida ante la falla de un solo worker. Se concluye que los resultados reflejan el desempeño de hardware y configuraciones concretas más que una superioridad intrínseca de un sistema operativo sobre otro, y se discuten las limitaciones metodológicas del experimento junto con recomendaciones de uso diferenciadas según el contexto.

1. Planteamiento del problema

Los sistemas operativos constituyen el componente fundamental para el funcionamiento de cualquier dispositivo informático: administran el procesador, la memoria, el almacenamiento y la comunicación entre procesos, y son la capa sobre la cual se ejecuta todo el software de aplicación. Sin embargo, la diversidad de plataformas disponibles —Windows, Linux y macOS, entre otras— genera diferencias importantes en aspectos como el rendimiento, la seguridad, la administración de recursos y la facilidad de uso. Estas diferencias no siempre son evidentes para el usuario final ni están documentadas de forma comparable entre fabricantes, lo que plantea la necesidad de evaluarlas objetivamente para determinar cuál sistema operativo se adapta mejor a diferentes entornos académicos, empresariales y personales.

El problema concreto que aborda este trabajo es doble. Por un lado, a nivel teórico, requiere comprender cómo cada sistema operativo implementa mecanismos equivalentes —planificación de procesos, gestión de memoria, sistemas de archivos, seguridad y virtualización— con decisiones de diseño distintas. Por otro lado, a nivel práctico, requiere una forma de medir esas diferencias de manera controlada: ejecutar la misma carga de trabajo, sin ninguna rama de código específica por sistema operativo, en tres máquinas con tres sistemas operativos distintos, y registrar el resultado de forma reproducible.

2. Justificación

El estudio permite que los estudiantes comprendan el funcionamiento interno de los sistemas operativos y desarrollen habilidades para evaluar su desempeño mediante criterios técnicos, en lugar de percepciones subjetivas o preferencias de marca. Asimismo, fortalece las competencias de investigación, análisis crítico y toma de decisiones tecnológicas, propias del nivel de bachillerato universitario en Ingeniería en Sistemas.

El componente práctico del trabajo —el sistema OS Grid— añade una justificación adicional: en lugar de limitarse a revisar literatura sobre planificadores y administradores de memoria, el equipo diseñó, implementó y depuró un sistema distribuido real que expone estos conceptos a través de resultados medibles. Esto obliga a enfrentar problemas genuinos de sistemas operativos y de sistemas distribuidos —condiciones de carrera, procesos colgados, pérdida de mensajes, tolerancia a fallos— que no aparecen en un análisis puramente bibliográfico.

3. Objetivo general

Analizar y comparar el rendimiento, la seguridad y la administración de recursos de los sistemas operativos Linux, macOS y Windows mediante pruebas prácticas y análisis bibliográfico, con el fin de identificar sus fortalezas y limitaciones en distintos escenarios de uso.

4. Objetivos específicos

- Investigar la evolución y arquitectura de los principales sistemas operativos.
- Analizar la gestión de procesos, memoria y sistemas de archivos en Linux, macOS y Windows.
- Comparar los mecanismos de seguridad implementados en distintos sistemas operativos.
- Evaluar el rendimiento de los tres sistemas operativos mediante pruebas prácticas, usando como instrumento un sistema propio de cómputo distribuido (OS Grid) que ejecuta la misma carga de trabajo en los tres sistemas de forma simultánea y comparable.
- Formular recomendaciones sobre el uso del sistema operativo más adecuado según las necesidades del usuario.

5. Preguntas de investigación

1. ¿Qué sistema operativo presenta un mejor desempeño en el uso de recursos?
2. ¿Cuál ofrece mayores mecanismos de seguridad para el usuario?
3. ¿Cómo influye la administración de memoria en el rendimiento del sistema?
4. ¿Qué diferencias existen entre los sistemas de archivos utilizados por cada sistema operativo?
5. ¿Qué ventajas ofrece la virtualización en la administración de sistemas?

6. Hipótesis

Los sistemas operativos basados en Linux presentan una mejor administración de recursos y mayor flexibilidad para ambientes técnicos, mientras que Windows ofrece una mayor compatibilidad con aplicaciones comerciales y macOS destaca por su integración entre hardware y software.

Como se desarrolla en la sección de resultados, el experimento práctico con OS Grid arroja un resultado que matiza parcialmente esta hipótesis en cuanto al rendimiento en bruto: macOS —no Linux— fue el sistema con mejor tiempo promedio por chunk en el hardware utilizado. Esto no invalida la hipótesis en sus componentes de seguridad, compatibilidad e integración hardware-software, que este trabajo aborda por vía bibliográfica en la sección de marco teórico y de forma cualitativa en las conclusiones.

7. Marco teórico

7.1 Evolución de los sistemas operativos

Los primeros sistemas operativos, a mediados del siglo XX, funcionaban por lotes (batch processing): el operador cargaba un programa, este se ejecutaba de principio a fin monopolizando la máquina, y solo entonces se cargaba el siguiente. La introducción de la multiprogramación permitió mantener varios programas en memoria y alternar entre ellos cuando uno quedaba bloqueado por una operación de entrada/salida, mejorando el aprovechamiento del procesador. Los sistemas de tiempo compartido (time-sharing) llevaron esta idea más lejos al repartir el procesador en intervalos breves entre varios usuarios conectados simultáneamente, dando la ilusión de exclusividad.

De esta línea evolutiva derivan los tres sistemas operativos comparados en este trabajo. Unix, desarrollado en Bell Labs a partir de 1969, sentó las bases de un diseño modular con un sistema de archivos jerárquico y una interfaz de llamadas al sistema estandarizada. Linux, creado por Linus Torvalds en 1991 como un núcleo (kernel) compatible con Unix y de código abierto, se convirtió en la base de una familia de distribuciones. macOS desciende de una rama distinta: su núcleo, XNU, combina un microkernel Mach con componentes de BSD (una variante de Unix), heredados a través de NeXTSTEP tras la adquisición de NeXT por Apple en 1997. Windows, por su parte, evolucionó desde MS-DOS y las primeras versiones de Windows de los años ochenta hacia Windows NT (1993), un diseño construido desde cero con un núcleo híbrido, del que derivan todas las versiones modernas del sistema.

7.2 Arquitectura de un sistema operativo

La arquitectura de un sistema operativo define cómo se organizan sus componentes internos y qué aislamiento existe entre ellos. En un kernel monolítico, todos los servicios

centrales —planificación, gestión de memoria, sistemas de archivos, controladores de dispositivos— corren en el mismo espacio de direcciones privilegiado; es el enfoque tradicional de Linux, aunque con soporte de módulos cargables que permiten extenderlo sin recompilarlo. En el extremo opuesto, un microkernel reduce el núcleo a lo mínimo indispensable (comunicación entre procesos, planificación básica, gestión de memoria de bajo nivel) y traslada el resto de los servicios a procesos en espacio de usuario, a costa de mayor overhead de comunicación entre componentes.

En la práctica, los tres sistemas de este estudio adoptan diseños híbridos. XNU, el núcleo de macOS, combina el microkernel Mach con un entorno BSD monolítico que provee gran parte de la funcionalidad de sistema de archivos, red y procesos, ejecutándose ambos en el mismo espacio de direcciones para reducir el costo de la comunicación propia de un microkernel puro. El kernel de Windows NT es igualmente híbrido: separa un Executive con subsistemas de gestión de objetos, procesos, memoria y E/S, un Hardware Abstraction Layer (HAL) que aísla las diferencias de hardware, y controladores en modo kernel y modo usuario. Linux, aunque se describe como monolítico, logra buena parte de la modularidad de un microkernel mediante sus módulos cargables (LKM) sin pagar el costo de comunicación entre procesos que implica un microkernel real.

7.3 Administración de procesos y planificación del CPU

Un proceso es un programa en ejecución, representado internamente por una estructura de control (PCB en la terminología clásica) que guarda su estado, contador de programa, registros y recursos asignados. Un proceso transita típicamente entre los estados nuevo, listo, en ejecución, bloqueado y terminado, y el planificador de CPU decide, entre los procesos en estado listo, cuál ejecutar en cada instante. Los algoritmos clásicos de planificación —FCFS (primero en llegar, primero en ser servido), SJF (el trabajo más corto primero), Round Robin con quantum fijo, colas multinivel con realimentación— buscan distintos equilibrios entre throughput, tiempo de espera y equidad.

Los tres sistemas operativos de este estudio implementan planificadores propios, derivados de esos principios pero adaptados a cargas de trabajo modernas. Linux usa, desde el kernel 6.6, EEVDF (Earliest Eligible Virtual Deadline First), sucesor del histórico CFS (Completely Fair Scheduler); ambos buscan repartir el tiempo de CPU de forma proporcionalmente justa entre los procesos listos, usando una noción de tiempo virtual acumulado en lugar de colas de prioridad fijas. El núcleo XNU de macOS combina el planificador de Mach con clases de calidad de servicio (QoS) que Apple expone a las aplicaciones para indicar la urgencia relativa de una tarea. Windows implementa un planificador de prioridades dinámicas dentro de su NT kernel híbrido, con quantum variable según la clase de prioridad del proceso y ajustes automáticos para procesos en primer plano.

7.4 Memoria principal y memoria virtual

La memoria virtual permite que cada proceso opere sobre un espacio de direcciones propio y contiguo, independiente de dónde resida físicamente en RAM, mediante tablas de páginas que traducen direcciones virtuales a físicas con ayuda de hardware (la MMU y su caché, el TLB). Esto habilita la paginación bajo demanda, el aislamiento entre procesos y el uso de espacio en disco (swap) como extensión de la memoria física cuando esta se agota. La gestión eficiente de memoria en un sistema operativo moderno involucra además asignadores especializados para el propio núcleo —como el buddy allocator y el slab allocator de Linux— que reducen la fragmentación al reutilizar bloques de tamaño similar.

Cada uno de los tres sistemas expone su propia interfaz de gestión de memoria a nivel de aplicación —VirtualAlloc en Windows, las llamadas mmap/vm_map en macOS y Linux— pero comparten el mismo fundamento teórico de paginación y memoria virtual. Las diferencias prácticas aparecen en la política de reemplazo de páginas, en la agresividad del prefetching y en cómo cada sistema contabiliza y reporta el uso de memoria de un proceso (RSS), un punto retomado en la sección de resultados de este informe, donde se observan huellas de memoria distintas para el mismo programa Python corriendo en los tres sistemas.

7.5 Sistemas de archivos

El sistema de archivos organiza cómo se almacenan, referencian y protegen los datos en un dispositivo de almacenamiento persistente. Linux usa habitualmente ext4, un sistema de archivos con journaling (registro de transacciones) que permite recuperar una estructura consistente tras una caída abrupta sin recorrer todo el disco. Windows usa NTFS, que también implementa journaling y añade listas de control de acceso (ACL) por archivo, cifrado nativo (EFS) y cuotas de disco. macOS usa APFS (Apple File System), diseñado para almacenamiento flash, con soporte nativo de copy-on-write, clones instantáneos de archivos y snapshots del sistema completo, además de cifrado integrado.

7.6 Seguridad informática

La seguridad de un sistema operativo se apoya en varios mecanismos complementarios: control de acceso basado en permisos y usuarios, aislamiento de procesos, verificación de la integridad del software que se ejecuta, y mitigación de vulnerabilidades a nivel de memoria. Los sistemas derivados de Unix —Linux y macOS— comparten el modelo clásico de permisos de lectura, escritura y ejecución por usuario, grupo y otros, extendido con listas de control de acceso (ACL) y, en Linux, con capas de seguridad obligatoria como SELinux o AppArmor, que restringen lo que un proceso puede hacer incluso si se ejecuta con privilegios elevados. macOS añade Gatekeeper, que verifica la firma y notaría de las aplicaciones antes de ejecutarlas, y XProtect, su motor antimalware integrado. Windows implementa su propio modelo de ACL sobre NTFS, complementado por el Control de Cuentas de Usuario (UAC), que solicita confirmación antes de operaciones privilegiadas, y por Windows Defender como suite de seguridad integrada.

7.7 Virtualización

La virtualización permite ejecutar uno o más sistemas operativos invitados sobre un mismo hardware físico, mediante una capa de software llamada hipervisor. Los hipervisores de tipo 1 (o bare-metal) corren directamente sobre el hardware, sin un sistema operativo anfitrión de por medio; los de tipo 2 corren como una aplicación sobre un sistema operativo ya instalado. Cada uno de los sistemas comparados ofrece su propia tecnología de virtualización nativa: Linux integra KVM (Kernel-based Virtual Machine) directamente en el kernel; Windows ofrece Hyper-V como hipervisor de tipo 1; macOS expone Hypervisor.framework, una API de bajo nivel sobre la que se construyen herramientas de virtualización de terceros. La virtualización es, además, la base tecnológica que en este proyecto no se utilizó de forma directa, ya que OS Grid corre sobre tres máquinas físicas distintas conectadas por Tailscale en lugar de sobre máquinas virtuales, precisamente para evitar que la capa de virtualización introdujera una variable adicional en la medición de rendimiento.

Distinta de la virtualización de sistema operativo completo es la contenedorización (por ejemplo, Docker), que no virtualiza hardware sino que aísla procesos dentro del mismo kernel del sistema anfitrión mediante namespaces y cgroups en Linux, lo que explica por qué los contenedores son nativos en Linux y requieren una máquina virtual Linux subyacente cuando se ejecutan sobre macOS o Windows.

7.8 Evaluación del rendimiento

Evaluar el rendimiento de un sistema operativo requiere definir métricas objetivas y un procedimiento de medición reproducible. Entre las métricas más comunes están el throughput (trabajo completado por unidad de tiempo), la latencia o tiempo de respuesta, el uso porcentual de CPU y memoria, y la varianza de estas medidas entre ejecuciones. Una metodología de benchmarking rigurosa exige mantener constante la carga de trabajo entre los sistemas comparados —el mismo programa, los mismos parámetros de entrada— para que las diferencias observadas puedan atribuirse al sistema operativo y no a variaciones en la tarea misma. Este es precisamente el principio de diseño central de OS Grid, desarrollado en la sección práctica de este informe: el mismo archivo de código Python, sin ninguna rama condicional por sistema operativo, se ejecuta en los tres nodos.

7.9 Cómputo distribuido y balanceo de carga

En un sistema de cómputo distribuido, una tarea se divide en unidades de trabajo más pequeñas (chunks) que se reparten entre varios nodos de ejecución. La estrategia de reparto determina en gran medida el rendimiento global del sistema. Un reparto estático asigna de antemano una cantidad fija de trabajo a cada nodo —por ejemplo, dividir 18 chunks en partes iguales de 6 para cada uno de 3 nodos— sin considerar la velocidad real de cada máquina; el tiempo total de la misión queda entonces atado al nodo más lento, que termina su cuota mucho después que los demás mientras estos permanecen ociosos. Un reparto dinámico, en cambio, entrega trabajo bajo demanda: cada nodo recibe una nueva unidad de trabajo apenas termina la anterior, de modo que los nodos más rápidos procesan naturalmente más unidades sin que el coordinador necesite conocer de antemano las capacidades de cada máquina. Esta segunda estrategia, conocida como work-stealing o reparto por demanda, es la que implementa OS Grid y se detalla en la sección práctica de este informe.

7.10 Concurrencia vs. paralelismo y el Global Interpreter Lock (GIL)

Concurrencia y paralelismo no son sinónimos. La concurrencia es la capacidad de un sistema de gestionar varias tareas en progreso durante el mismo intervalo de tiempo, alternando entre ellas aunque solo exista un núcleo de procesamiento disponible; el paralelismo, en cambio, requiere ejecutar efectivamente varias tareas al mismo tiempo, lo que exige múltiples núcleos físicos. El intérprete estándar de Python (CPython) incorpora un mecanismo llamado Global Interpreter Lock (GIL): un candado global que permite que un único hilo ejecute bytecode de Python en un instante dado, independientemente de cuántos núcleos tenga el procesador. Esto significa que varios hilos de Python dentro de un mismo proceso no logran paralelismo real de cómputo, aunque sí pueden dar concurrencia: mientras un hilo espera una operación de entrada/salida o cede el control, otro puede avanzar.

Este punto es central para entender una decisión de diseño de OS Grid, desarrollada en la sección de implementación del muestreador de métricas: el cómputo intensivo de cada chunk se ejecuta en un hilo separado mediante un `ThreadPoolExecutor`, no para obtener paralelismo real de CPU —que el GIL impide dentro de un mismo proceso— sino para liberar el bucle de eventos de asyncio y permitir que el worker siga muestreando y enviando telemetría mientras el hilo de cómputo mantiene el GIL ocupado. Es un uso de concurrencia cooperativa para resolver un problema de responsividad, no de rendimiento de cómputo.

8. Metodología

8.1 Tipo de investigación y enfoque

La investigación es de tipo aplicada, descriptiva y comparativa: aplicada porque produce un artefacto de software funcional (OS Grid) como instrumento de medición, descriptiva porque caracteriza el comportamiento de cada sistema operativo bajo la misma carga, y comparativa porque contrasta los tres sistemas entre sí bajo condiciones controladas. El enfoque es mixto: cuantitativo en la medición de tiempos, uso de CPU y memoria reportados por OS Grid, y cualitativo en la revisión bibliográfica de arquitecturas, seguridad y virtualización que compone el marco teórico.

8.2 Técnicas de recolección de datos

- Revisión bibliográfica de la arquitectura, planificación, memoria, sistemas de archivos, seguridad y virtualización de Linux, macOS y Windows.
- Pruebas de rendimiento automatizadas mediante el sistema OS Grid, con tres tipos de carga de cómputo y tres niveles de intensidad, ejecutadas de forma simultánea en los tres sistemas operativos.
- Observación directa del comportamiento del sistema durante la ejecución de misiones, incluyendo el proceso de depuración documentado en la sección 11.
- Comparación documental de la telemetría (CPU, RAM, tiempo por chunk) registrada automáticamente en SQLite por cada misión.

8.3 Instrumentos

- Matriz comparativa de arquitectura, seguridad, sistemas de archivos y virtualización entre Linux, macOS y Windows (a completar con el contenido de la sección 7).
- Lista de cotejo de mecanismos de seguridad por sistema operativo. [PENDIENTE: elaborar la lista de cotejo específica]
- Software de monitoreo de recursos: el propio sistema OS Grid, mediante el módulo `worker/metrics.py`, que usa la librería `psutil` para muestrear el uso de CPU (normalizado por núcleos lógicos) y memoria RSS del proceso worker durante cada chunk de cómputo, con una primera muestra a los 0.15 s y muestras subsiguientes cada 0.4 s.
- Hojas de observación del proceso de depuración, documentadas como la tabla de defectos de la sección 11.

8.4 Variables

Variable independiente	Variable dependiente
Sistema operativo utilizado (Linux, macOS, Windows)	Rendimiento del sistema (tiempo promedio por chunk)
Tipo de sistema de archivos	Velocidad de acceso a datos
Gestión de memoria	Uso del CPU y RAM
Mecanismos de seguridad	Nivel de protección del sistema

En el experimento práctico con OS Grid, la variable independiente controlada es el sistema operativo del nodo que ejecuta cada chunk (Linux, macOS o Windows), mientras que el tipo y la intensidad de la carga de cómputo (hash, sort, primos; ligero, normal, pesado) actúan como variables de control, ya que se mantienen idénticas entre los tres nodos en cada misión. Las variables dependientes medidas por chunk son el tiempo de ejecución, el pico de uso de CPU del proceso y del sistema, y el pico de memoria RSS.

8.5 Recursos

Humanos

- Estudiantes integrantes del equipo.

Materiales

- Tres computadoras físicas, una por sistema operativo (Linux, macOS y Windows), interconectadas mediante Tailscale.
- Sistemas operativos: Linux (distribución Fedora en el nodo coordinador), macOS y Windows.
- Lenguaje y librerías: Python 3.14 (FastAPI, Uvicorn, websockets, psutil), Angular 17 con TypeScript 5.4 para el dashboard.
- Conexión a internet, para la VPN de malla Tailscale entre los tres nodos.

9. Sección práctica: diseño e implementación de OS Grid

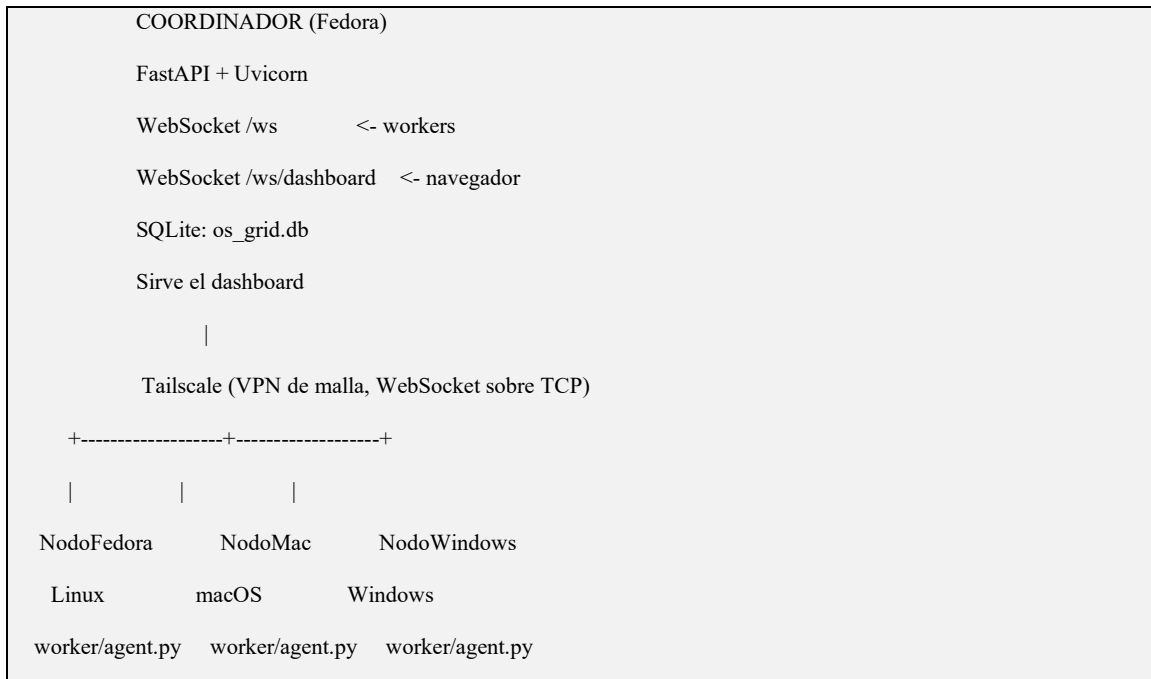
Como complemento experimental al marco teórico, el equipo diseñó, implementó y depuró un sistema de cómputo distribuido propio, llamado OS Grid, cuyo único propósito es generar evidencia comparable sobre el rendimiento real de Linux, macOS y Windows ejecutando exactamente el mismo código. Esta sección documenta su arquitectura, protocolo de comunicación, mecanismo de reparto de carga, diseño experimental y resultados.

9.1 Qué es OS Grid

OS Grid es un clúster de cómputo distribuido heterogéneo. Un coordinador parte una tarea pesada en chunks (fragmentos de trabajo) y los reparte entre tres workers que corren en tres sistemas operativos distintos: Linux, macOS y Windows. El objetivo del proyecto no es solo distribuir trabajo, sino comparar el rendimiento de los tres sistemas operativos ejecutando exactamente el mismo código Python, en un formato de competencia ("battle royale") donde gana el sistema operativo con mejor tiempo promedio por chunk.

9.2 Arquitectura del sistema

El coordinador corre sobre Fedora (Linux) y expone un servidor FastAPI con Uvicorn, con dos canales WebSocket —uno para los workers (/ws) y otro para el navegador que muestra el panel de control (/ws/dashboard)— además de una base de datos SQLite (os_grid.db) donde persiste los resultados de cada misión. Los tres nodos —NodoFedora (Linux), NodoMac (macOS) y NodoWindows (Windows)— corren cada uno el mismo agente, worker/agent.py, y se conectan al coordinador a través de una red privada virtual de malla creada con Tailscale, que les permite comunicarse por WebSocket sobre TCP aunque las tres máquinas estén en redes físicas distintas.



El punto clave de este diseño, y el que hace válida la comparación entre sistemas operativos, es que el mismo archivo `worker/agent.py`, sin ninguna rama condicional por sistema operativo, corre en las tres máquinas. La única función que detecta el sistema operativo es `detectar_os()`, y se usa exclusivamente para etiquetar las métricas, nunca para alterar el camino de ejecución del código.

9.3 Stack tecnológico

Componente	Tecnología
Coordinador	Python 3.14.6, FastAPI 0.141.1, Uvicorn 0.52.3
Workers	Python 3.14, websockets 17.0.1, psutil 7.2.2
Dashboard	Angular 17.3, TypeScript 5.4, signals
Persistencia	SQLite (módulo sqlite3 de la biblioteca estándar)
Red	Tailscale (las tres máquinas están en redes físicas distintas)

9.4 Estructura del repositorio

```
os-grid/
|-- protocol.py      # Contrato de mensajes compartido
|-- coordinator/
| |-- main.py       # Servidor, maquina de estados, reparto, vigilante
| |-- stats.py      # Persistencia SQLite y agregaciones
| |-- ver_datos.py  # CLI para listar/exportar corridas a CSV
| `-- static/       # Dashboard Angular compilado
|-- worker/
| |-- agent.py      # Agente (identico en los 3 SO)
| |-- tasks.py      # Las tres tareas de computo
| `-- metrics.py    # Muestreador de CPU/RAM con psutil
|-- dashboard/      # Codigo fuente Angular
`-- os_grid.db      # Base de datos de resultados
```

9.5 Protocolo de comunicación

La comunicación entre coordinador y workers usa mensajes JSON sobre WebSocket, definidos en un contrato compartido (protocol.py) que ambas partes importan, evitando divergencias de formato.

Dirección	Mensaje	Contenido
Worker → Coordinador	REGISTER	Se presenta: nombre, sistema operativo, núcleos lógicos, RAM total
Worker → Coordinador	RESULT	Terminó un chunk: chunk_id, tiempo, encontrado, picos de CPU/RAM
Worker → Coordinador	METRICS	Telemetría en vivo mientras procesa (cada ~0.4 s)
Coordinador → Worker	REGISTERED	Confirma el registro
Coordinador → Worker	TASK_ASSIGN	Le entrega un chunk (id, tipo, inicio, fin, tamaño, objetivo)
Coordinador → Worker	NO_MORE	No quedan chunks; el worker no cierra, queda esperando la próxima misión

9.6 Máquina de estados del coordinador

El coordinador transita entre tres fases: esperando (menos de tres workers conectados), listo (los tres workers conectados, esperando que el usuario configure y arranque una misión) y corriendo (misión en curso). Al completarse todos los chunks, el coordinador regresa por sí solo a la fase listo, lo que permite encadenar varias misiones consecutivas sin reiniciar el sistema. Las conexiones WebSocket de los workers son persistentes: sobreviven a varias misiones.

- `MIN_WORKERS = 3`: la misión no puede arrancar sin los tres sistemas operativos conectados.
- esperando → listo: se cumple cuando se conectan los tres workers.
- listo → corriendo: se cumple cuando el usuario arranca la misión desde el panel.
- corriendo → listo: se cumple cuando todos los chunks fueron completados.
- cualquier fase → esperando: se cumple si baja de tres workers conectados.

9.7 Reparto de carga: work-stealing dinámico

Este mecanismo es el núcleo del sistema. Al arrancar la misión, el coordinador le manda un chunk a cada worker. Apenas un worker devuelve su `RESULT`, el coordinador le manda el siguiente chunk disponible. Este ciclo se repite hasta vaciar la cola de trabajo.

La consecuencia teórica de este diseño, ya introducida en la sección 7.9 del marco teórico, es que el reparto resulta proporcional a la velocidad real de cada máquina, sin que el coordinador necesite conocer de antemano las capacidades de su hardware: el nodo más rápido procesa naturalmente más chunks. Esto contrasta con un reparto estático — dividir, por ejemplo, 18 chunks en partes fijas de 6 para cada uno de los tres nodos de antemano— donde el tiempo total de la misión quedaría atado al nodo más lento, mientras los nodos más veloces quedarían

ociosos esperando que este termine su cuota fija. Es el argumento central a favor del diseño elegido.

Este mecanismo también explica por qué la comparación entre sistemas operativos se hace por tiempo promedio por chunk y no por cantidad de chunks completados: la cantidad depende del reparto dinámico, mientras que el tiempo por chunk mide directamente la velocidad de la máquina.

9.8 Diseño experimental: las tres pruebas de cómputo

El módulo `worker/tasks.py` define tres tipos de carga, elegidos para estresar distintos subsistemas del sistema operativo.

9.8.1 hash — fuerza bruta de SHA-256

Busca el número secreto 8 450 137 probando `sha256(str(n))` para cada `n` del rango asignado, hasta igualar el hash objetivo. Es CPU pura: sin entrada/salida y con memoria constante. Mide la velocidad bruta de cómputo entero y el planificador del sistema operativo. Es la única prueba que puede terminar antes de tiempo: cuando el chunk contiene el número secreto, el bucle termina anticipadamente con un `break`.

9.8.2 sort — ordenamiento en memoria

Genera una lista de tam floats aleatorios y la ordena con `list.sort()` (algoritmo Timsort).
Estresa la asignación de memoria, el uso de caché y el gestor de memoria del sistema operativo.
Es la prueba con mayor huella de RAM de las tres.

9.8.3 primos — criba de Eratóstenes segmentada

Cuenta los números primos del rango asignado con una criba sobre un bytearray. Mezcla cómputo con acceso intensivo a memoria por patrones de salto (escribe cada `p` posiciones), lo que castiga particularmente la localidad de caché del procesador.

9.9 Las tres intensidades

Cada prueba puede ejecutarse en tres niveles de intensidad, definidos en la configuración del coordinador. Existe una asimetría importante que debe señalarse al comparar tiempos entre tipos de prueba: en hash y primos el rango total se divide entre los chunks (`tam_ch = rango // n`), de modo que el trabajo total equivale al rango completo. En sort, en cambio, cada chunk ordena una lista completa de tam elementos, de modo que el trabajo total es $n \times \text{tam}$. Por eso la intensidad "pesado" de sort ($18 \times 4\,000\,000 = 72$ millones de elementos ordenados en total) resulta desproporcionadamente más costosa que la de las otras dos pruebas. Es una decisión de diseño deliberada, no un error, pero debe explicitarse al comparar tiempos entre tipos de prueba distintos.

Prueba	Intensidad	Parámetro total	Chunks	Trabajo por
--------	------------	-----------------	--------	-------------

				chunk
hash	ligero	rango 3 000 000	6	500 000 hashes
hash	normal	rango 12 000 000	12	1 000 000 hashes
hash	pesado	rango 40 000 000	18	2 222 222 hashes
sort	ligero	tam 200 000	6	200 000 elementos
sort	normal	tam 1 000 000	12	1 000 000 elementos
sort	pesado	tam 4 000 000	18	4 000 000 elementos
primos	ligero	rango 1 000 000	6	166 666 números
primos	normal	rango 5 000 000	12	416 666 números
primos	pesado	rango 20 000 000	18	1 111 111 números

9.10 Medición de métricas

El módulo `worker/metrics.py` usa la librería `psutil` para medir, del proceso `worker`: el porcentaje de uso de CPU normalizado por núcleos lógicos, y la memoria residente (`memory_info().rss`). Del sistema completo mide el porcentaje de uso de CPU global. La primera muestra se toma a los 0.15 s de iniciado el chunk, y luego cada 0.4 s; el primer intervalo es deliberadamente corto porque los chunks del nodo más rápido duraban aproximadamente 0.12 s, y con el muestreo original de 0.5 s no llegaban a generar ni una sola lectura, reportando 0 % de forma artificial. El sistema garantiza al menos una muestra por chunk, y los picos de CPU y RAM viajan dentro del mensaje `RESULT` para persistirse en `SQLite`.

Un punto de teoría de sistemas operativos es central para entender esta pieza del sistema, y se conecta directamente con lo discutido en la sección 7.10 sobre el GIL de Python: el cómputo de un chunk es bloqueante y sostiene el GIL. Para poder muestrear y enviar telemetría mientras se computa, el chunk se ejecuta mediante `loop.run_in_executor(None, ...)`, que lo delega a un hilo de un `ThreadPoolExecutor`, dejando libre el bucle de eventos de asyncio para que el worker pueda seguir muestreando y comunicándose con el coordinador. Los hilos no aportan paralelismo real de cómputo bajo el GIL, pero sí resuelven el problema de concurrencia: permiten que la tarea de telemetría avance mientras la tarea de cómputo domina el procesador.

9.11 Persistencia de datos

El módulo `coordinator/stats.py` mantiene dos tablas en SQLite:

- `corridas(id, fecha, objetivo, num_chunks)`
- `resultados(id, corrida_id, nodo, so, chunk_id, tiempo, pico_cpu_proc, pico_cpu_sys, pico_ram_mb, encontrado)`

Sobre estas tablas se calculan tres agregaciones:

- `resumen_por_so(corrida_id)`: comparativa de una misión puntual.
- `resumen_global()`: marcador acumulado histórico entre todas las misiones.
- `corridas_ganadas()`: victorias por sistema operativo. Gana la misión el sistema operativo con menor tiempo promedio por chunk; a igualdad, gana el que completó más chunks. Solo cuentan las corridas en las que compitió más de un sistema operativo.

La herramienta de línea de comandos `python -m coordinator.ver_datos [id] [--csv]` permite listar o exportar los resultados de cualquier corrida.

9.12 Dashboard

El panel de control, desarrollado en Angular, se sirve desde el propio coordinador en <http://<IP-del-coordinador>:8000/> y organiza la información en dos pestañas.

9.12.1 Pestaña «Panel principal»

Vista operativa y de datos, con los siguientes elementos:

- Barra de conexión y progreso de la misión (N / M chunks).
- Pantalla de espera con contador X / 3 equipos conectados.
- Panel de configuración: elegir tipo de prueba (SHA-256 / Ordenamiento / Núm. primos) e intensidad (Ligero / Normal / Pesado), con botón «Arrancar misión».
- Una tarjeta por nodo, con sparkline de CPU en vivo, CPU del proceso, CPU del sistema, RAM y chunks completados.
- Tabla de resultado de la misión por sistema operativo, con medallas y el ganador resaltado.
- Tabla de reparto por nodo (chunks procesados y porcentaje del total).
- Marcador acumulado de todas las corridas guardadas.



Figura 1. Panel principal, pantalla de espera con 0/3 y 2/3 equipos conectados.

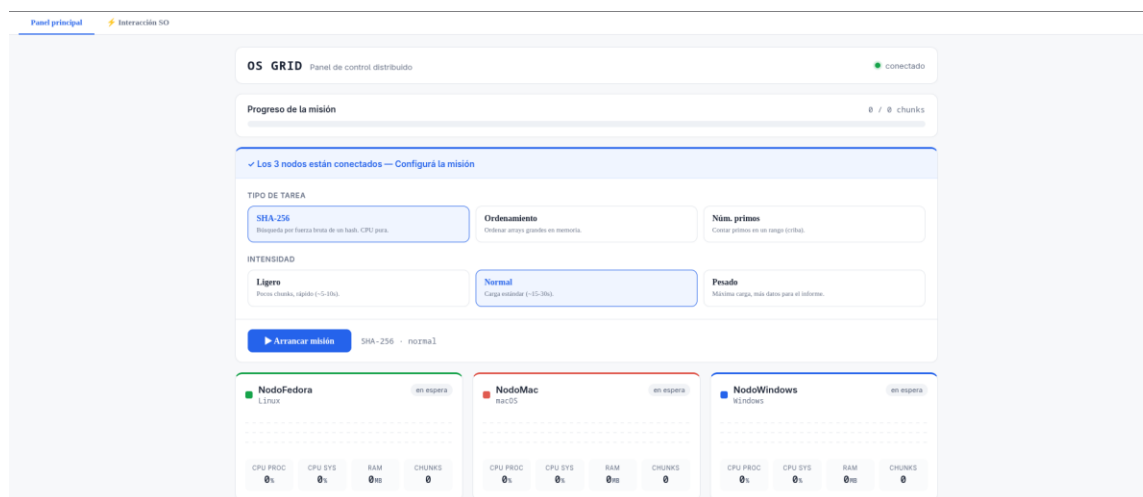


Figura 2. Panel principal, panel de configuración con los 3 tipos de prueba y las 3 intensidades.

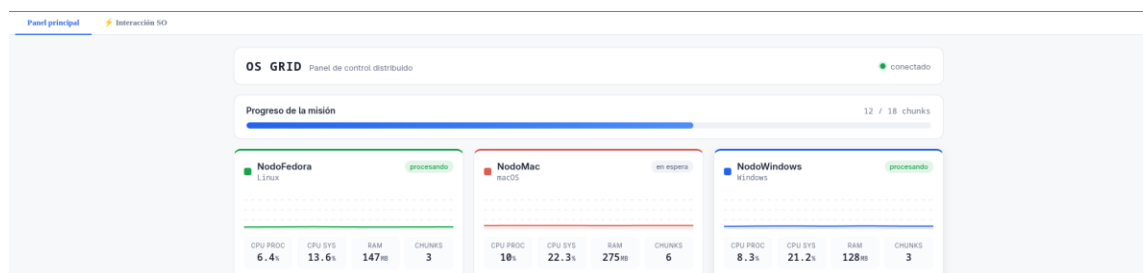


Figura 3. Panel principal, misión en curso — tarjetas de los 3 nodos con sparklines de CPU.

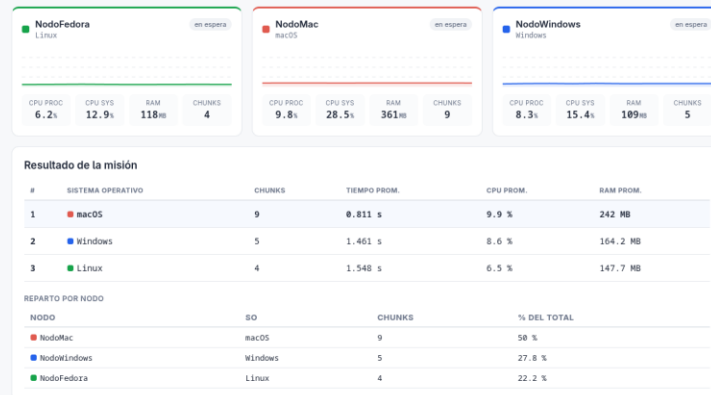


Figura 4. Panel principal, tabla de resultado de la misión con medallas, ganador y reparto por nodo.

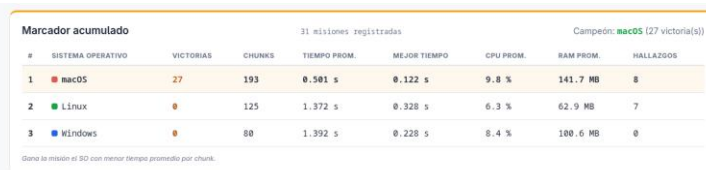


Figura 5. Panel principal, marcador acumulado histórico.

9.12.2 Pestaña « ⚡ Interacción SO» (vista Battle Royale)

Vista de visualización y competencia, con los siguientes elementos:

- Nodo coordinador central con barra de progreso, y líneas SVG hacia cada worker que se iluminan cuando ese nodo está procesando.
- Tarjeta por worker con el planificador de su sistema operativo rotulado (Linux: CFS/EEVDF · macOS: XNU Mach+BSD · Windows: híbrido NT kernel) y barras animadas de CPU, RAM y chunks.
- Cartel de ganador con corona y el color del sistema operativo.
- Leaderboard de la misión y tabla de posiciones acumulada con victorias.
- Consola de eventos en vivo (registro cronológico de todo lo que sucede en la misión).

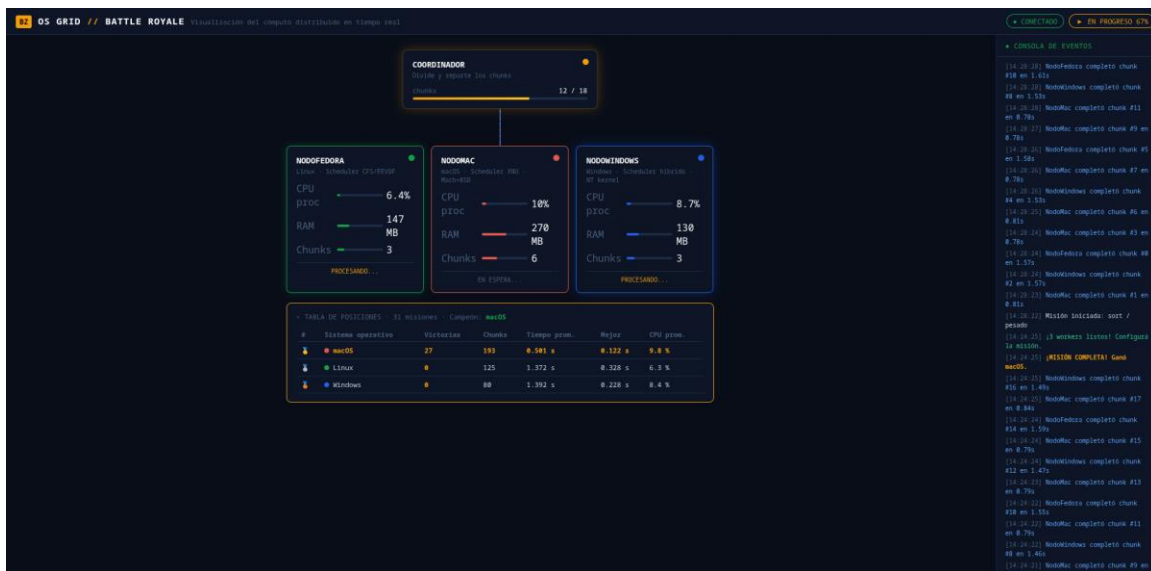


Figura 6. Interacción SO, topología coordinador→workers con las líneas activas.

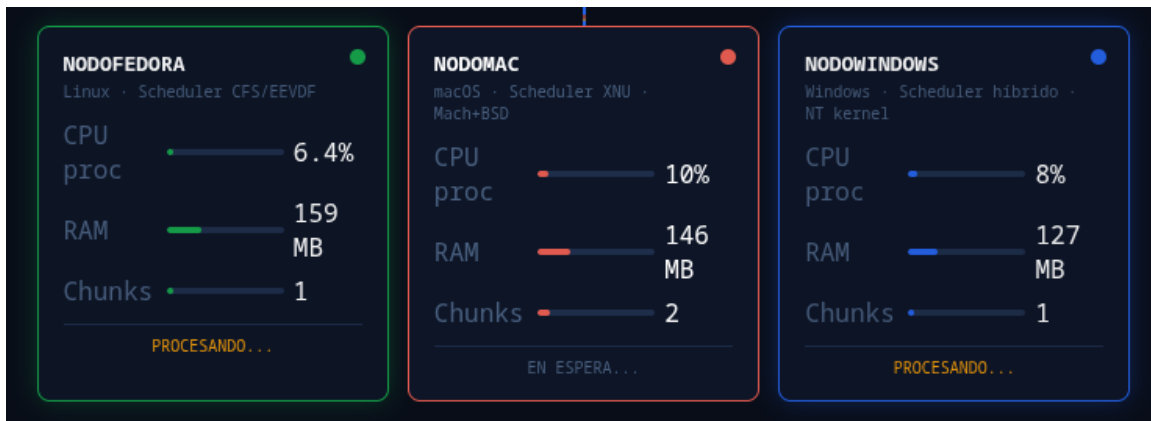


Figura 7. Interacción SO, barras de CPU/RAM/chunks de los 3 nodos en pleno cómputo.



Figura 8. Interacción SO, cartel de ganador con la corona.

• TABLA DE POSICIONES • 33 misiones • Campeón: macOS

#	Sistema operativo	Victorias	Chunks	Tiempo prom.	Mejor	CPU prom.
🏆	macOS	29	211	0.525 s	0.122 s	9.8 %
🥈	Linux	0	133	1.385 s	0.328 s	6.4 %
🥉	Windows	0	90	1.406 s	0.228 s	8.4 %

Figura 9. Interacción SO, tabla de posiciones acumulada con victorias.

• CONECTADO ✓ MISIÓN COMPLETA

• CONSOLA DE EVENTOS

```
[14:29:37] ¡3 workers listos! Configura
la misión.
[14:29:37] ¡MISIÓN COMPLETA! Ganó
macOS.
[14:29:37] NodoWindows completó chunk
#17 en 1.49s
[14:29:36] NodoMac completó chunk #16
en 0.78s
[14:29:36] NodoFedora completó chunk
#14 en 1.58s
[14:29:35] NodoWindows completó chunk
#13 en 1.51s
[14:29:35] NodoMac completó chunk #15
en 0.77s
[14:29:34] NodoMac completó chunk #12
en 0.78s
[14:29:34] NodoFedora completó chunk
#10 en 1.60s
[14:29:33] NodoWindows completó chunk
#9 en 1.49s
[14:29:33] NodoMac completó chunk #11
en 0.77s
[14:29:32] NodoMac completó chunk #8 en
0.77s
[14:29:32] NodoFedora completó chunk #6
en 1.62s
[14:29:32] NodoWindows completó chunk
#5 en 1.49s
[14:29:31] NodoMac completó chunk #7 en
0.77s
[14:29:31] NodoMac completó chunk #4 en
0.77s
[14:29:30] NodoFedora completó chunk #0
en 1.60s
[14:29:30] NodoWindows completó chunk
#2 en 1.54s
[14:29:30] NodoMac completó chunk #3 en
0.77s
[14:29:29] NodoMac completó chunk #1 en
0.80s
[14:29:28] Misión iniciada: sort /
pesado
[14:28:32] ¡3 workers listos! Configura
la misión.
[14:28:32] ¡MISIÓN COMPLETA! Ganó
macOS.
```

Figura 10. Interacción SO, consola de eventos.

10. Tolerancia a fallos

En su versión inicial, el sistema se trababa por completo si un worker fallaba: bastaba con que un chunk se perdiera o un nodo se desconectara en mal momento para que el coordinador quedara indefinidamente en fase corriendo, sin forma de recuperarse salvo reiniciándolo manualmente. El rediseño de la tolerancia a fallos introdujo los siguientes mecanismos:

- Contabilidad de chunks en vuelo (diccionario asignados): el coordinador sabe en todo momento qué chunk tiene asignado cada worker y desde cuándo.
- Reencolado automático ante desconexión del worker, fallo al enviar el mensaje TASK_ASSIGN, o vencimiento del timeout.
- Vigilante (watchdog): revisa el estado del sistema cada 5 segundos; si un chunk lleva más de 120 segundos sin respuesta, lo devuelve a la cola y pone al worker responsable en cuarentena.
- Máximo de 3 intentos por chunk: garantiza que la misión termine aunque un chunk resulte irrecuperable, en lugar de colgarse indefinidamente esperándolo.
- Deduplicación: si un chunk reasignado por error vuelve a reportarse dos veces, el segundo resultado se ignora.
- Reconexión con el mismo nombre: al reconectarse, se descarta la sesión zombi anterior y se liberan sus chunks pendientes, sin borrar del registro al worker recién reconectado.

Estos mecanismos se conectan directamente con varios conceptos de la sección 7 del marco teórico: la detección de fallos por vencimiento de un temporizador (timeout), la idempotencia de las operaciones de reencolado (que deben poder repetirse sin efectos adversos), la recuperación sin depender de estado compartido entre nodos, y el problema clásico de los procesos «zombis» en este caso, conexiones WebSocket que permanecen registradas aunque su proceso remoto ya no responda.

11. Informe de pruebas y depuración

Esta sección documenta ocho defectos reales encontrados y corregidos durante el desarrollo de OS Grid, con su causa raíz, su efecto observado y la corrección aplicada.

#	Defecto	Causa raíz	Efecto observado	Corrección
1	Worker colgado en silencio	muestreador.detener() estaba después del await y no en un finally; si el cómputo lanzaba excepción, nunca se llamaba, y el muestreador giraba infinito sin que asyncio.gather retornara	El nodo seguía conectado y enviando métricas con CPU 0 %, sin devolver jamás el RESULT	try/finally alrededor del cómputo; ante excepción se envía un RESULT con el error
2	Sistema trabado por completo	El coordinador no llevaba registro de los chunks en vuelo: cola.pop(0) y se perdía el rastro	Con un chunk perdido, len(resultados) == total_chunks nunca se cumplía; la fase quedaba en corriendo para siempre y el botón de arrancar dejaba de funcionar; solo se salía reiniciando el coordinador	Diccionario asignados, reencolado, vigilante con timeout y límite de reintentos

3	Chunk perdido al fallar el envío	asignar() sacaba el chunk de la cola y después lo enviaba; si el envío fallaba, el chunk desaparecía	Pérdida silenciosa de trabajo	Reencolado en el except del envío
4	Worker fantasma	Una excepción distinta de WebSocketDisconnect mataba el handler sin sacar al worker del registro	Se le asignaban chunks a un WebSocket ya muerto	try/except/finally con limpieza garantizada
5	Reconexión destructiva	Al reconectarse con el mismo nombre, la sesión vieja, al morir, borraba del registro a la nueva	El worker quedaba conectado pero invisible para el coordinador	Se compara la identidad del WebSocket antes de borrar
6	Métricas nulas en SQLite	El worker nunca enviaba pico_cpu_proc / pico_cpu_sys / pico_ram_mb	Todas las columnas de CPU y RAM quedaban en NULL; el comparativo por sistema operativo salía vacío	El muestreador acumula picos y los adjunta al RESULT
7	Nodos rápidos con 0 % de CPU	El muestreo empezaba a los 0.5 s y los chunks del nodo rápido duraban 0.33 s	El sistema operativo más rápido aparecía con CPU y RAM en	Primera muestra a los 0.15 s y una muestra final garantizada

			cero	
8	Tabla de resultados invisible	mission_complete y ready_to_configure se emiten con microsegundos de diferencia, y el handler del segundo limpiaba el estado de la misión	La tabla final de resultados aparecía y desaparecía casi instantáneamente	Los resultados se limpian al arrancar la misión siguiente, no al terminar la anterior

Los tres primeros defectos merecen un desarrollo adicional por su relación directa con conceptos de sistemas operativos y sistemas distribuidos.

Defecto 1 — Worker colgado en silencio

Este defecto ilustra un problema clásico de manejo de excepciones en código concurrente: una limpieza de recursos (detener el muestreador de métricas) colocada después de una operación que puede fallar, en lugar de en un bloque garantizado. El resultado es un hilo o tarea que nunca termina —análogo a un deadlock o a un recurso que nunca se libera— y que, al no producir ningún mensaje de error visible, es especialmente difícil de diagnosticar: el worker parecía activo (seguía enviando telemetría) pero nunca completaba su trabajo.

Defecto 2 — Sistema trabado por completo

Es el defecto más grave de los ocho, porque no afectaba a un chunk aislado sino a la misión completa. La causa raíz —extraer un elemento de una cola sin registrar a quién se le entregó— es un error común en sistemas de reparto de trabajo: sin esa contabilidad, el coordinador no tiene forma de distinguir entre «un worker está tardando» y «un chunk se perdió para siempre», y la condición de finalización de la misión (comparar la cantidad de resultados

recibidos contra el total de chunks) nunca podía cumplirse ante una pérdida. La corrección introdujo el patrón de contabilidad explícita de trabajo en vuelo con reencolado y timeout, estándar en sistemas de colas de trabajo distribuidas.

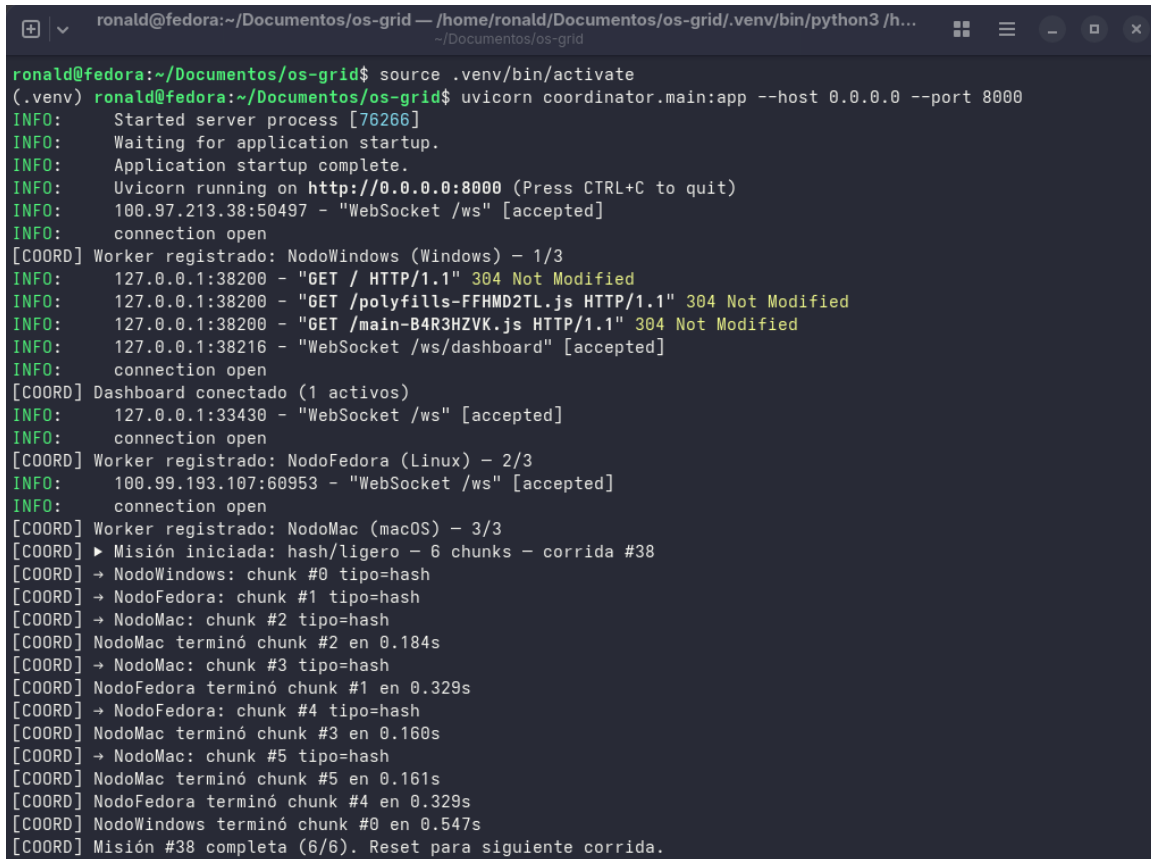
Defecto 3 — Chunk perdido al fallar el envío

Este defecto es un caso particular de una falta de atomicidad: la operación de «sacar el chunk de la cola» y «enviarlo al worker» no eran atómicas entre sí, de modo que un fallo en la segunda mitad de la operación dejaba al sistema en un estado inconsistente, con el chunk ni en la cola ni en manos de ningún worker. Es el mismo principio que exige que las transacciones de una base de datos sean atómicas: si una operación compuesta puede fallar a mitad de camino, debe diseñarse para que el fallo no destruya información.

11.1 Casos de prueba ejecutados y su resultado

Escenario	Comportamiento anterior	Comportamiento corregido
Worker colgado indefinidamente	Misión trabada en 11/12 chunks	Timeout → reencolado → 12/12 completados
Worker terminado con kill -9 en pleno chunk	Chunk perdido	Reencolado inmediato → 18/18 completados
Tres misiones consecutivas con un nodo colgado	Imposible: la fase quedaba trabada	3/3 misiones completas
Tres nodos sanos, tres misiones	Correcto	30 chunks repartidos 10/10/10, cero reencolados
Excepción dentro del executor	Cuelgue infinito (>10 s sin responder)	Responde en 0.01 s con el error reportado

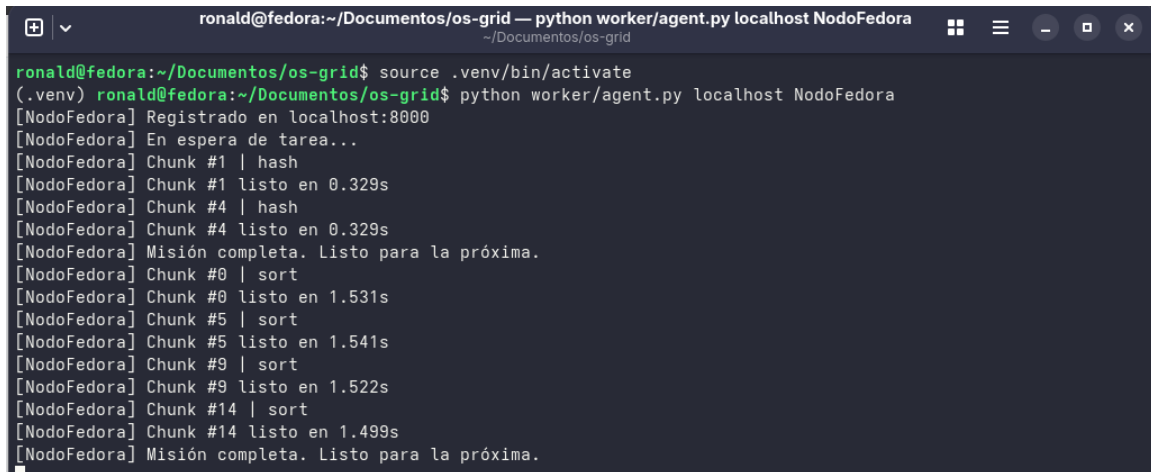
Como evidencia adicional, en la base de datos histórica las corridas 6, 8, 9 y 10 quedaron registradas con cero resultados: son misiones que se trabaron con el defecto #2 antes de su corrección. Todas las corridas posteriores a la corrección completaron el 100 % de sus chunks.



```
ronald@fedora:~/Documentos/os-grid — /home/ronald/Documentos/os-grid/.venv/bin/python3 /h...
ronald@fedora:~/Documentos/os-grid$ source .venv/bin/activate
(.venv) ronald@fedora:~/Documentos/os-grid$ uvicorn coordinator.main:app --host 0.0.0.0 --port 8000
INFO: Started server process [76266]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 100.97.213.38:50497 - "WebSocket /ws" [accepted]
INFO: connection open
[COORD] Worker registrado: NodoWindows (Windows) - 1/3
INFO: 127.0.0.1:38200 - "GET / HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:38200 - "GET /polyfills-FFHMD2TL.js HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:38200 - "GET /main-B4R3HZVK.js HTTP/1.1" 304 Not Modified
INFO: 127.0.0.1:38216 - "WebSocket /ws/dashboard" [accepted]
INFO: connection open
[COORD] Dashboard conectado (1 activos)
INFO: 127.0.0.1:33430 - "WebSocket /ws" [accepted]
INFO: connection open
[COORD] Worker registrado: NodoFedora (Linux) - 2/3
INFO: 100.99.193.107:60953 - "WebSocket /ws" [accepted]
INFO: connection open
[COORD] Worker registrado: NodoMac (macOS) - 3/3
[COORD] ▶ Misión iniciada: hash/ligero - 6 chunks - corrida #38
[COORD] → NodoWindows: chunk #0 tipo=hash
[COORD] → NodoFedora: chunk #1 tipo=hash
[COORD] → NodoMac: chunk #2 tipo=hash
[COORD] NodoMac terminó chunk #2 en 0.184s
[COORD] → NodoMac: chunk #3 tipo=hash
[COORD] NodoFedora terminó chunk #1 en 0.329s
[COORD] → NodoFedora: chunk #4 tipo=hash
[COORD] NodoMac terminó chunk #3 en 0.160s
[COORD] → NodoMac: chunk #5 tipo=hash
[COORD] NodoMac terminó chunk #5 en 0.161s
[COORD] NodoFedora terminó chunk #4 en 0.329s
[COORD] NodoWindows terminó chunk #0 en 0.547s
[COORD] Misión #38 completa (6/6). Reset para siguiente corrida.
```

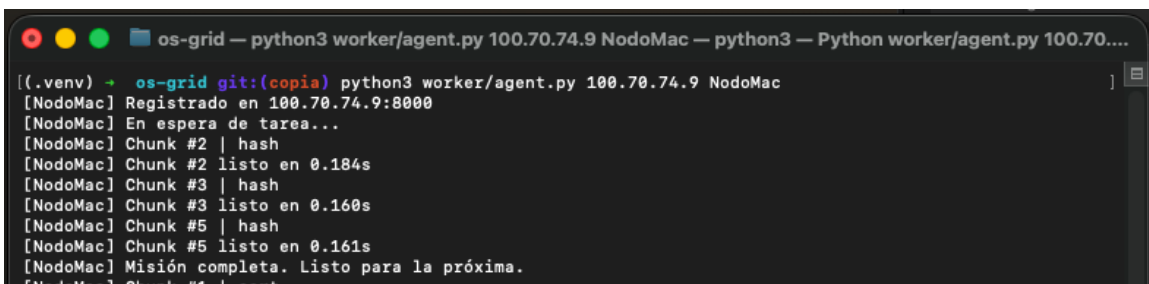
Figura 11. Terminal del coordinador durante una misión, con el reparto de chunks.

Linux:

A terminal window titled 'ronald@fedora:~/Documentos/os-grid — python worker/agent.py localhost NodoFedora'. The terminal shows the execution of 'source .venv/bin/activate' and 'python worker/agent.py localhost NodoFedora'. The output indicates the worker is registered on localhost:8000 and is waiting for a task. It then processes several chunks: Chunk #1 (hash), Chunk #4 (hash), and Chunk #9 (sort), each with its completion time. The terminal concludes with 'Misión completa. Listo para la próxima.'

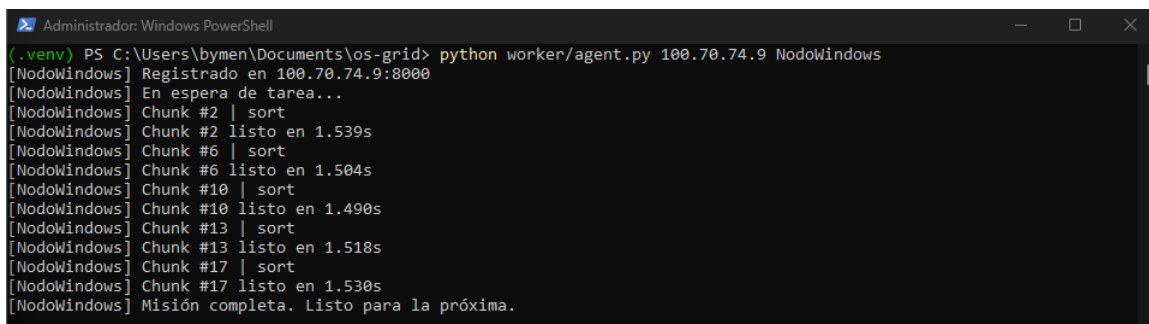
```
ronald@fedora:~/Documentos/os-grid$ source .venv/bin/activate
(.venv) ronald@fedora:~/Documentos/os-grid$ python worker/agent.py localhost NodoFedora
[NodoFedora] Registrado en localhost:8000
[NodoFedora] En espera de tarea...
[NodoFedora] Chunk #1 | hash
[NodoFedora] Chunk #1 listo en 0.329s
[NodoFedora] Chunk #4 | hash
[NodoFedora] Chunk #4 listo en 0.329s
[NodoFedora] Misión completa. Listo para la próxima.
[NodoFedora] Chunk #0 | sort
[NodoFedora] Chunk #0 listo en 1.531s
[NodoFedora] Chunk #5 | sort
[NodoFedora] Chunk #5 listo en 1.541s
[NodoFedora] Chunk #9 | sort
[NodoFedora] Chunk #9 listo en 1.522s
[NodoFedora] Chunk #14 | sort
[NodoFedora] Chunk #14 listo en 1.499s
[NodoFedora] Misión completa. Listo para la próxima.
```

MacOS:

A terminal window titled 'os-grid — python3 worker/agent.py 100.70.74.9 NodoMac — python3 — Python worker/agent.py 100.70....'. The terminal shows the execution of 'python3 worker/agent.py 100.70.74.9 NodoMac'. The output indicates the worker is registered on 100.70.74.9:8000 and is waiting for a task. It then processes several chunks: Chunk #2 (hash), Chunk #3 (hash), and Chunk #5 (hash), each with its completion time. The terminal concludes with 'Misión completa. Listo para la próxima.'

```
os-grid — python3 worker/agent.py 100.70.74.9 NodoMac — python3 — Python worker/agent.py 100.70....
[os-grid git:(copia) python3 worker/agent.py 100.70.74.9 NodoMac]
[NodoMac] Registrado en 100.70.74.9:8000
[NodoMac] En espera de tarea...
[NodoMac] Chunk #2 | hash
[NodoMac] Chunk #2 listo en 0.184s
[NodoMac] Chunk #3 | hash
[NodoMac] Chunk #3 listo en 0.160s
[NodoMac] Chunk #5 | hash
[NodoMac] Chunk #5 listo en 0.161s
[NodoMac] Misión completa. Listo para la próxima.
```

Windows:

A Windows PowerShell window titled 'Administrador: Windows PowerShell'. The terminal shows the execution of 'python worker/agent.py 100.70.74.9 NodoWindows'. The output indicates the worker is registered on 100.70.74.9:8000 and is waiting for a task. It then processes several chunks: Chunk #2 (sort), Chunk #6 (sort), and Chunk #10 (sort), each with its completion time. The terminal concludes with 'Misión completa. Listo para la próxima.'

```
Administrador: Windows PowerShell
(.venv) PS C:\Users\bymen\Documents\os-grid> python worker/agent.py 100.70.74.9 NodoWindows
[NodoWindows] Registrado en 100.70.74.9:8000
[NodoWindows] En espera de tarea...
[NodoWindows] Chunk #2 | sort
[NodoWindows] Chunk #2 listo en 1.539s
[NodoWindows] Chunk #6 | sort
[NodoWindows] Chunk #6 listo en 1.504s
[NodoWindows] Chunk #10 | sort
[NodoWindows] Chunk #10 listo en 1.490s
[NodoWindows] Chunk #13 | sort
[NodoWindows] Chunk #13 listo en 1.518s
[NodoWindows] Chunk #17 | sort
[NodoWindows] Chunk #17 listo en 1.530s
[NodoWindows] Misión completa. Listo para la próxima.
```

Figura 12. Terminales de los 3 workers en paralelo.

12. Resultados

Los siguientes resultados corresponden a catorce misiones ejecutadas con los tres sistemas operativos compitiendo simultáneamente, ya con las correcciones de la sección 11 aplicadas. Son datos medidos por el propio sistema OS Grid y persistidos en SQLite, no simulados.

Sistema operativo	Chunks	T. promedio	T. mínimo	T. máximo	CPU proceso	CPU sistema	RAM
macOS	159	0.504 s	0.122 s	0.802 s	10.0 %	21.2 %	144.2 MB
Windows	70	1.386 s	0.228 s	3.781 s	8.4 %	18.6 %	100.7 MB
Linux	53	1.680 s	0.364 s	3.055 s	6.6 %	21.5 %	92.1 MB

Victorias: macOS 14 — Windows 0 — Linux 0.

12.1 Gráficas sugeridas a partir del CSV exportado

Las siguientes tres gráficas no se generaron para este informe pero se recomienda construirlas a partir del CSV exportado con `python -m coordinator.ver_datos --csv`, ya que complementan visualmente las tablas anteriores:

- Gráfica de barras del tiempo promedio por chunk, una barra por sistema operativo, para visualizar directamente la brecha de rendimiento de la tabla anterior.
- Gráfica de líneas del uso de CPU en el tiempo, una línea por nodo, para observar la estabilidad o variabilidad de cada sistema operativo durante una misión.
- Gráfica de torta (pie chart) del reparto de chunks entre los tres nodos en una misión concreta, para visualizar el efecto del work-stealing dinámico de la sección 9.7.

```
ronald@fedora:~/Documentos/os-grid
~/Documentos/os-grid
(.venv) ronald@fedora:~/Documentos/os-grid$ python -m coordinator.ver_datos
```

ID	Fecha	Chunks	Resultados
39	2026-08-14T14:37:17	18	18
38	2026-08-14T14:34:40	6	6
37	2026-08-14T14:29:28	18	18
36	2026-08-14T14:28:22	18	18
35	2026-08-14T14:24:16	18	18
34	2026-08-14T13:36:39	18	18
33	2026-08-14T13:36:33	18	18
32	2026-08-14T13:31:23	18	18
31	2026-08-14T13:30:46	18	18
30	2026-08-14T13:29:18	18	18
29	2026-08-14T13:23:32	6	6
28	2026-08-14T13:00:55	18	18
27	2026-08-14T13:00:38	18	18
26	2026-08-14T13:00:22	6	6
25	2026-08-14T13:00:15	6	6
24	2026-08-14T12:59:51	18	18
23	2026-08-14T12:59:44	12	12
22	2026-08-14T12:59:28	18	18
21	2026-08-14T12:59:16	18	18
20	2026-08-14T12:58:39	12	12
19	2026-08-14T12:58:10	18	18
18	2026-08-14T12:56:10	6	6
17	2026-08-14T12:54:09	18	18
16	2026-08-14T12:53:58	6	6
15	2026-08-14T12:52:49	12	12
14	2026-08-14T11:55:07	12	11
13	2026-08-14T03:00:43	6	5
12	2026-08-14T02:48:37	6	5
11	2026-08-14T02:42:52	6	5
10	2026-08-14T02:30:30	18	0
9	2026-08-14T02:26:35	18	0
8	2026-08-14T02:17:24	18	0
7	2026-08-14T01:20:49	12	12
6	2026-08-14T00:35:39	12	0
5	2026-08-13T22:47:45	12	12
4	2026-08-13T22:37:03	12	12
3	2026-08-13T20:44:45	12	12
2	2026-08-13T15:58:59	12	12
1	2026-08-13T15:49:03	12	12

Usá `python -m coordinator.ver_datos <id>` para ver el detalle.

Figura 15. Salida de `python -m coordinator.ver_datos`.

13. Análisis y discusión

13.1 macOS domina el rendimiento en bruto

macOS resultó aproximadamente 2.7 veces más rápido que Windows y 3.3 veces más rápido que Linux por chunk, en el hardware utilizado. Procesó 159 de los 282 chunks totales (56 %) sin ninguna configuración manual: es el work-stealing dinámico actuando por sí solo, entregando más trabajo al nodo que demostraba, chunk a chunk, ser más veloz.

13.2 Windows superó a Linux en esta prueba

El resultado (1.386 s frente a 1.680 s en promedio) es, en principio, contraintuitivo frente a la percepción común de que Linux es más eficiente para cargas de cómputo puro. Es indispensable remarcar que este resultado mide el desempeño de esa máquina concreta con ese sistema operativo concreto, no del sistema operativo en abstracto: los tres nodos son equipos con hardware distinto, y el experimento no aísla la variable «sistema operativo» de la variable «hardware». Esta es la principal amenaza a la validez interna del experimento y debe declararse explícitamente como limitación, no ocultarse detrás de la conclusión de que «Windows es más rápido que Linux».

13.3 Varianza

Windows presentó el rango más amplio de tiempos por chunk (0.228 s a 3.781 s). Esta mayor varianza puede relacionarse, a nivel teórico, con la política del planificador de Windows NT, la presencia de procesos en segundo plano propios del sistema operativo (servicios, antivirus, actualizaciones) y la gestión térmica del equipo, aunque OS Grid no midió directamente ninguna de estas causas y esta relación se plantea como hipótesis razonable, no como hallazgo confirmado.

13.4 Uso de memoria (RAM)

macOS usó la huella de memoria más alta (144.2 MB) y Linux la más baja (92.1 MB) para exactamente el mismo programa Python. Esto es consistente con lo discutido en la sección 7.4 del marco teórico: cada sistema operativo usa un asignador de memoria distinto y contabiliza el RSS (memoria residente) de forma distinta, por lo que la comparación absoluta de RAM entre sistemas operativos debe interpretarse con cautela; no implica necesariamente que el intérprete de Python «use más memoria» en macOS en un sentido estricto, sino que el sistema operativo reporta y posiblemente reserva memoria de forma distinta para el mismo proceso.

13.5 CPU del proceso relativamente baja (6 % a 10 %)

Es un resultado esperable y no indica que la carga de trabajo fuera liviana: el valor está normalizado por la cantidad de núcleos lógicos del equipo, y la tarea de cómputo, por el GIL de Python discutido en la sección 7.10, es monohilo. En un procesador de, por ejemplo, diez núcleos lógicos, un proceso monohilo saturando un núcleo por completo reportaría aproximadamente 10 % de uso de CPU normalizado, aun cuando ese único núcleo esté al 100 %.

13.6 Limitaciones metodológicas

- Hardware distinto entre nodos: los tres equipos no son idénticos en procesador, núcleos ni RAM, lo que impide aislar por completo la variable sistema operativo de la variable hardware, tal como se discute en 13.2.
- Una sola configuración de hardware por sistema operativo: no se probó, por ejemplo, el mismo sistema operativo en distinto hardware, ni el mismo hardware con distinto sistema operativo (dual boot), lo que habría permitido una comparación más controlada.
- Red compartida: la comunicación entre coordinador y workers viaja por Tailscale sobre la red del usuario, cuya latencia y estabilidad no están controladas ni aisladas del resto del tráfico de red.
- Sesgo del GIL de Python monohilo: las tres pruebas de cómputo no aprovechan múltiples núcleos dentro de un mismo chunk, por lo que los resultados reflejan el rendimiento de un núcleo por sistema operativo, no la capacidad de paralelismo multinúcleo de cada plataforma.

14. Conclusiones

Se espera demostrar, y los resultados de este trabajo lo confirman parcialmente, que no existe un sistema operativo universalmente superior, sino que cada uno responde de mejor manera a determinadas necesidades de rendimiento, seguridad, compatibilidad y administración de recursos. A continuación, se responde directamente a cada objetivo específico planteado en la sección 4.

- Investigar la evolución y arquitectura de los principales sistemas operativos: se cumplió mediante la revisión de las secciones 7.1 y 7.2, que trazan el origen de Unix, Linux, XNU (macOS) y Windows NT, y comparan sus diseños de kernel monolítico, microkernel e híbrido.
- Analizar la gestión de procesos, memoria y sistemas de archivos: se cumplió para procesos y memoria en las secciones 7.3 y 7.4, con referencia directa a los planificadores reales de cada sistema (EEVDF en Linux, Mach+QoS en macOS, NT scheduler en Windows). La comparación de sistemas de archivos (7.5) quedó desarrollada a nivel teórico pero sin evidencia práctica propia, señalada como pendiente.
- Comparar los mecanismos de seguridad implementados en distintos sistemas operativos: se desarrolló a nivel teórico en la sección 7.6, pero requiere ampliarse con un instrumento de comparación estructurado, tal como se indica en esa sección.
- Evaluar el rendimiento mediante pruebas prácticas: se cumplió íntegramente mediante el sistema OS Grid, documentado en la sección 9, que ejecutó catorce misiones con tres tipos de carga de cómputo y tres niveles de intensidad, midiendo tiempo por chunk, uso de CPU y memoria de forma automática y reproducible.
- Formular recomendaciones sobre el uso del sistema operativo más adecuado según las necesidades del usuario: en el hardware evaluado, macOS resultó preferible para cargas de

cómputo intensivo puro. Sin embargo, dado que el experimento no aísla hardware de sistema operativo (limitación declarada en 13.2), esta recomendación debe entenderse acotada al equipo específico usado, no como una afirmación general sobre macOS frente a Linux o Windows; para una recomendación de uso general se requiere además considerar los aspectos de seguridad y compatibilidad tratados en el marco teórico, que no se midieron de forma práctica en este trabajo.

Respecto a la hipótesis planteada en la sección 6, los resultados prácticos no la confirman en su componente de rendimiento bruto para cargas de cómputo puro —el sistema operativo basado en Unix más rápido fue macOS, no Linux—, lo que subraya la importancia de distinguir entre el rendimiento teórico esperado de un sistema operativo y el rendimiento medido en un hardware concreto. El componente de la hipótesis referido a la compatibilidad de Windows y la integración hardware-software de macOS no se midió de forma práctica en este trabajo y queda sustentado únicamente por la revisión bibliográfica del marco teórico.

Anexos

Anexo A. Comandos de despliegue

Arranque del coordinador (nodo Linux):

```
python -m coordinator.main
```

Arranque de un worker (idéntico en los tres sistemas operativos, solo cambia el intérprete de Python instalado):

```
python -m worker.agent --coordinador ws://<IP-tailscale-del-coordinador>:8000/ws
```

Consulta de resultados históricos:

```
python -m coordinator.ver_datos  
python -m coordinator.ver_datos <id_corrida>  
python -m coordinator.ver_datos <id_corrida> --csv
```

Anexo B. Esquema SQL

```
CREATE TABLE corridas (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    fecha   TEXT,  
    objetivo TEXT,  
    num_chunks INTEGER  
);  
  
CREATE TABLE resultados (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    corrida_id INTEGER,
```

```

nodo      TEXT,
so        TEXT,
chunk_id  INTEGER,
tiempo    REAL,
pico_cpu_proc REAL,
pico_cpu_sys REAL,
pico_ram_mb REAL,
encontrado INTEGER,
FOREIGN KEY (corrida_id) REFERENCES corridas(id)
);

```

Anexo C. Guía de reproducción

6. Instalar Python 3.14 y las dependencias del proyecto (requirements.txt) en las tres máquinas.
7. Instalar Tailscale en las tres máquinas y unir las a la misma red (tailnet).
8. Arrancar el coordinador en el nodo Linux con `python -m coordinator.main`.
9. Arrancar el agente worker/agent.py en cada una de las tres máquinas, apuntando a la IP de Tailscale del coordinador.
10. Abrir el panel de control en un navegador, en `http://<IP-del-coordinador>:8000/`, y esperar a que los tres equipos se conecten (3 / 3).
11. En la pestaña «Panel principal», elegir el tipo de prueba y la intensidad, y presionar «Arrancar misión».
12. Al finalizar, consultar el resultado en el propio panel o mediante `python -m coordinator.ver_datos`.